Cheat Sheet POO

Concepte generale

Modificatori de acces = cuvinte rezervate ce controlează accesul celorlalte clase la membrii unei clase.

Specificator	Clasa	Subcls*	Pachet	Oriunde
Private	X			
Implicit	X		X	
Protected	X	X	X	
Public	X	X	X	X

StringBuffer

- Este asemănător unui obiect String doar că este modificabil în sensul că există metode pentru modificarea lungimii şi a conţinutului său – **append, insert**.

Adăugarea:
StringBuffer append(String str)
StringBuffer append(int num)
Inserarea se face într-o anumită poziţie.
StringBuffer insert(int index, String str)
StringBuffer append(int index, char ch)

- **delete()** - şterge un subşir. Subşirul începe în poziţia specificată şi se termină în poziţia index de final - 1 sau până la sfârşitul şirului dacă nu se specifică poziţie de final.

public StringBuffer **delete**(int start, int end)
public StringBuffer **delete**(int start)

- **replace()** – Înlocuieşte un subşir din şirul sursă cu un alt şir

public StringBuffer **replace**(int start, int end, String str)

- **substring()**

public String substring(int start)

- **reverse()** – reversul şirului (ordine inversă a caracterelor)

public StringBuffer **reverse**()

- **length()**

public int **length**()

- **capacity() –** Returnează capacitatea curentă a StringBuffer-ului. Capacitatea este dimensiunea disponibilă pentru adăugarea de noi elemente

public int **capacity**()

- **charAt()**

public char **charAt**(int index)

- **getChars()**

public void **getChars**(int srcBegin, int srcEnd, char[] dst, int dstBegin)

- **setLength**() **–** Setează lungimea pentru obiectul de tip StringBuffer.

public void **setLength**(int newLength)

String vs StringBuffer

String -> **Immutable** = unui obiect de tip String nu i se poate modifica conținutul! Se crează un nou obiect String modificat!

StringBuffer -> **Mutable** = unui obiect de tip StringBuffer i se poate modifica conținutul

StringTokenizer

- utilizată pentru împărțirea în atomi lexicali (tokens)

public **StringTokenizer**(String str, String delim)
public boolean **hasMoreTokens**()
public String **nextToken**()

Exemplu:

```
StringTokenizer st = new StringTokenizer("this, is a
   test.“,” ,.”);
while (st.hasMoreTokens()) {
   System.out.println(st.nextToken());
}
```

Principii POO

1. ***Abstractizarea - Abstraction***

2. ***Încapsularea - Encapsulation***
   - ascunderea de informații (**data-hiding**)
   - doar metodele proprii ale obiectului pot accesa starea acestuia
   - separă comportarea de structură, definită prin implementare

3. ***Modularizarea***
   - procesul de partiționare a unui program în component individuale (module)

Formatted: Font: (Default) Comfortaa, 17 pt, Bold, Italic
Formatted: Font: (Default) Comfortaa, 17 pt, Bold, Italic
Formatted: Indent: Left: 1,9 cm
Formatted: Indent: Left: 1,27 cm
Formatted: Indent: Left: 1,9 cm
Formatted: Font: 11 pt

- modulele servesc ca și containere în care sunt declarate clasele și iobiectele programului

4. ***Poliformism - Polymorphism***

- mai multe funcții pot avea **același nume** în același domeniu de definiție, dacă se pot diferenția prin **numărul** sau **tipul argumentelor** de apel

O funcţie este polimorfică dacă se poate executa cu acelaşi efect asupra unor valori de tipuri diferite (ex. operatorul & din C)

- o funcție este **supraîncărcată (overloading)** dacă execută operații diferite în contexte diferite (ex. operator + din Java) (supraîncărcarea se poate aplica doar funcțiilor)

Nu este admis ca funcţiile să difere doar prin tipul returnat!

- **supradefinirea (overriding)** oferă posibilitatea de a redefini metode pentru **clasele derivate**, metodele au **același tip și aceeași parametri**

Supradefinire

- metodele **declarate public** în superclasă trebuie să fie obligatoriu **publice** în toate subclasele
- metodele **declarate protected** în superclasă trebuie să fie **publice sau protected** în subclase (nu pot fi private sau implicit)
- metodele declarate private în superclasă nu se moștenesc, așa că pentru ele nu există reguli de supradefinire
- metodele declarate fără modificator de acces (implicit) în superclasă trebuie să fie declarate public, protected sau implicit în subclase (nu pot fi private)

Regulă:
Putem supradefini cu un nivel de acces mai mare sau egal, dar nu mai mic (weaker)!

5. ***Ierarhizarea***

- modalitatea de a ordona abstractizările (tipurile abstracte de date)
- ierarhiile pot să denote **relații de tip** (**moștenirile** între clase) sau relații de **agregare** (**compunerea unui obiect** din mai multe obiecte mai simple)

6. ***Moștenirea - Inheritance***

- permite construirea unor clase noi, care păstrează caracteristicile şi comportarea, deci datele şi funcţiile membru, de la una sau mai multe clase definite anterior, numite **clase de bază**, fiind posibilă redefinirea sau adăugarea unor date şi funcţii noi
- o clasă ce moștenește una sau mai multe clase de bază se numește clasă derivată

Garbage Collector

- procesul responsabil cu eliberarea memoriei

System.gc
"Sugerează" JVM să elibereze memoria

Finalizarea

- **Metoda finalize este apelată automat înainte de eliminarea unui obiect din memorie.**

finalize ≠ destructor

Constructori

- **this** apelează explicit un **constructor al clasei**

Formatted: Indent: Left: 1,27 cm
Formatted: Indent: Left: 1,9 cm
Formatted: Font: 11 pt

```
class Dreptunghi {
    double x, y, w, h;
    Dreptunghi(double x1, double y1, double w1, double
      h1) {
      // Implementam doar constructorul cel mai general
      x=x1; y=y1; w=w1; h=h1;
      System.out.println("Instantiere dreptunghi");
    }
    Dreptunghi(double w1, double h1) {
      this(0, 0, w1, h1);
      // Apelam constructorul cu 4 argumente
    }
    Dreptunghi() {
      this(0, 0);
      // Apelam constructorul cu 2 argumente
    }
}
```

- **super** apelează explicit un **constructor al superclasei**

```
class Patrat extends Dreptunghi {
    Patrat(double x, double y, double d) {
        super(x, y, d, d);
    }
    Patrat(double d){
      // super(d,d);   sau:
      this(0,0,d);
    }
}
```

Apelul explicit al unui constructor nu poate apărea decât într-un alt constructor şi trebuie să fie prima instrucţiune din constructorul respectiv.

Lanțul de apeluri al constructorilor

- cazul în care o clasă de bază este extinsă (moștenită) de către o clasă copil
- **constructorii nu se moștenesc!**
- constructorii pot fi invocați implicit sau explicit (super)
- ori de câte ori este creat un obiect din clasa copil, este **invocat mai întâi constructorul (fără parametri) al clasei părinte**

Trimiterea parametrilor (1)

TipReturnat metoda([Tip1 arg1, Tip2 arg2, ...])

- **Argumentele sunt trimise doar prin valoare (pass-by-value).**

```
void metoda(StringBuffer sir, int numar) {
    // StringBuffer este tip referinta
    // int este tip primitiv
    sir.append("abc");
    numar = 123;
}
...
StringBuffer s=new StringBuffer();
int n=0;
metoda(s, n);
System.out.println(s + ", " + n);
// s va fi "abc", dar n va fi 0
```

Clase Wrapper

byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

- toate clasele wrapper sunt **immutable** => de fiecare dată când se **modifică** valoarea unui obiect wrapper este **creat un obiect nou**

Variabile de instanță VS variabile de clasă

```
class Exemplu {
   int x ; //variabila de instanta
}
```

- **variabilă de instanță** = la fiecare creare a unui obiect al clasei Exemplu sistemul alocă **o zonă de memorie separată** pentru memorarea valorii lui x
- **variabilă de clasă** - sistemul alocă o singură zonă de memorie la care au acces toate instanțele clasei respective => **dacă un obiect modifică valoarea unei variabile statice ea se va modifica și pentru celelalte obiecte**

Formatted: Font: 11 pt

```
class Exemplu {
   static int sx ; //variabila de clasă
}
```

- Deoarece nu depind de o anumită instanţă a unei clase, variabilele statice pot fi referite şi sub forma:

  NumeClasa.numeVariabilaStatica

  Ex. Exemplu.sx

```
class Exemplu {
    int x ; // Variabila de instanta
    static long n; // Variabila de clasa
}
    ...
    Exemplu o1 = new Exemplu();
    Exemplu o2 = new Exemplu();
    o1.x = 100;
    o2.x = 200;
    System.out.println(o1.x); // Afiseaza 100
    System.out.println(o2.x); // Afiseaza 200
    o1.n = 100;
    System.out.println(o2.n); // Afiseaza 100
    o2.n = 200;
    System.out.println(o1.n); // Afiseaza 200
    System.out.println(Exemplu.n); // Afiseaza 200
    // o1.n, o2.n si Exemplu.n sunt referinte la aceeasi
        // valoare
```

Metode de instanță VS metode de clasă

- **metodele de instanță** operează atât **variabilele de instanță** cât și pe cele statice ale clasei
- **metodele de clasă** operează doar **variabilele statice** ale clasei

```
class Exemplu {
    int x ;      // Variabilă de instanţă
    static long n; // Variabilă de clasă
    void metodaDeInstanta() {
        n ++; // Corect
        x --; // Corect
    }
    static void metodaStatica() {
        n ++; // Corect
        x --; // Eroare la compilare !
    }
}
```

Intocmai ca şi la variabilele statice, întrucât metodele de clasă nu depind de starea obiectelor clasei respective, apelul lor se poate face şi sub forma:

NumeClasa.numeMetodaStatica

- **metodele de instanță** nu pot fi apelate decât pentru un obiect al clasei respective

```
Exemplu.metodaDeInstanta(); // Eroare
Exemplu obj = new Exemplu();
obj.metodaDeInstanta(); // Corect
```

Agregare VS Compunere

- prezența unei referințe către un obiect dintr-o clasă într-o altă classic

Agregare (aggregation) - obiectul container poate exista și în absența obiectelor agregate (care pot fi null)

Compunerea (composition) - existența unui obiect este dependentă de un alt obiect. La dispariția obiectelor conținute, existența obiectului container încetează

Blocuri statice de inițializare

```
static {
    // Bloc static de initializare;
    ...
}
```

Blocuri de inițializare

```
{
    //  cod de initializare
}
```

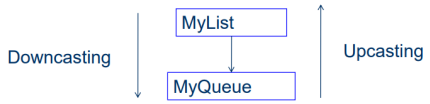
- **blocurile statice de inițializare** ale clasei se execută la un moment care **precede prima utilizare activă a clasei respective**

Deleted: clasă

Formatted: Font: 11 pt

- compilatorul Java copiază **blocurile de inițializare** în fiecare **constructor**

Upcasting vs Downcasting



Upcasting = convertirea unei referințe de la o **clasă derivată** într-una a unei **clase de bază**

- este **realizat automat** (nu trebuie declarat explicit)

```
class Patrat extends Poligon { … }

Poligon metoda1( ) {
    Poligon p = new Poligon();
    Patrat t = new Patrat();
    if (...)  return p; // Corect
    return t; // upcast automat!!! – corect!
}
```

Downcasting - operația inversă upcasting-ului

- conversie **explicită** de tip în care se merge în **jos** pe ierarhia claselor (se convertește o clasă de bază într-una derivată)
- trebuie făcută **explicit** conversia

```
boolean equals(Object o) {
    if (!(o instanceof Car)) return false;
    Car other = (Car) o;
    // compare this to other and return
}
```

- este posibil numai dacă obiectul declarat ca fiind de o clasă de bază este, de fapt, o **instanță a clasei derivate** către care se face downcasting (altfel se va arunca o excepție la rularea programului)

Excepții

- o eroare este o excepție, dar o excepție nu reprezintă neapărat o eroare de programare
- clasa care definește obiectul trebuie sa **mostenească**, direct sau indirect, clasa predefinita **Throwable.** In practica, la definirea excepțiilor, NU se extinde direct aceasta clasa, ci se utilizează **clasa Exception**, o subclasă a clasei Throwable.

in

Prinderea și tratarea excepțiilor
Bloc try-catch-finally

- în secțiunea **try** se introduce codul pe parcursul căruia pot să apară excepții
- în fiecare secțiune **catch** se amplaseaza secvența de instrucțiuni ce trebuie sa se execute in momentul in care secțiunea in try a apărut o excepție de tipul parametrului secțiunii catch (sau un subtip)
- In secțiunea **finally**, se amplaseaza cod ce trebuie neapărat sa se execute înaintea părăsirii blocului try-catch-finally, indiferent dacă a apărut sau nu vreo excepție (tratată sau netratată)

```
try - catch - finally
try {
    … // Instrucţiuni care pot genera excepţii
}
catch (TipExceptie1 variabila) {
    … // Tratarea excepţiilor de tipul 1
}
catch (TipExceptie2 variabila) {
    … // Tratarea excepţiilor de tipul 2
}
. . .
finally {
    … // Cod care se execută indiferent
    … // dacă apar sau nu excepţii
}
```

Formatted: Font: Bold

Formatted: Font: 11 pt

```
try {
        //Secventa de instructiuni in care ar putea sa
        //apara exceptii
} chatch(TipExceptie1 e) {
        //Secventa de instructiuni ce se executa cand, in
        //sectiunea try apare o exceptie, avand tipul sau
        //subtipul TipExceptie1 (parametrul fiind o referinta
        //la exceptia prinsa)
} catch(TipExceptie2 e) {
        //Acelasi lucru doar ca este vorba de TipExceptie2
}
//Sectiunea catch poate lipsi din acest bloc
//.....................................................
catch (TipExceptiN e) {
        //Acelasi lucru doar ca este vorba de TipExceptien
} finaly {
        //Secventa de instructiuni ce se executa in orice
        //conditii la terminarea executiei celorlalte sectiuni
        //Aceasta este obtionala (poate exista maxim una)
}
//Alte instructiuni ale metodei
```

Aruncarea excepțiilor

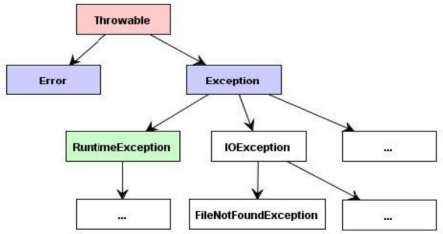
- Clauza **throws** apare in **antetul** unui metode și ne spune ce tipuri de excepții pot conduce la terminarea neobișnuită a respectivei metode
- după clauza throws se pot introduce mai multe nume de excepții, separate prin virgula

```
[modific] TipReturnat metoda([argumente])
   throws TipExceptie1, TipExceptie2, ...
{
...
}
```

Avantajele tratării excepțiilor

1. Separarea codului
2. Propagarea erorilor
3. Gruparea erorilor după tip

Ierarhia claselor ce descriu excepții



Crearea propriilor excepții

```
public class ExceptieProprie extends
   Exception {
    public ExceptieProprie(String mesaj) {
       super(mesaj);
       /* Apeleaza constructorul superclasei
         Exception */
    }
}
```

ArithmeticException

- împărțirea la 0 va genera o excepție doar dacă tipul numerelor impartite este **aritmetic întreg**
- în cazul tipurilor reale (float şi double) nu va fi generată nici o excepţie, ci va fi furnizat ca rezultat o constantă care poate fi, funcţie de operaţie, Infinity, -Infinity, sau NaN

Clase abstracte și interfețe

Commented [1]: subiect teorie restanta 2019

Clase abstracte

- clase pentru care programatorul nu intenționează să instanțieze obiecte (**nu se pot crea instanțe ale unei clase abstracte**)

Ex. Number nr=new Number(); // NU
Number nr=new Integer(): // DA

- clase de bază în ierarhii

Clasă abstractă

```
[public] abstract class ClasaAbstracta ... {
   // Declaratii uzuale
   // Declaratii de metode abstracte
}
```

Metodă abstractă

```
abstract class ClasaAbstracta {
   abstract void metodaAbstracta(); // Corect
   void metoda(); // Eroare
}
```

- o **metodă abstractă** nu poate apărea decât într-o **clasă abstractă**
- orice clasă care are o metodă abstractă trebuie declarată ca fiind abstractă
- *o clasă abstractă poate să nu aibă nicio metodă abstractă!*

Clasă abstractă -> interfață comună, funcționalitate diferită pentru fiecare subtip

Formatted: Font: 11 pt

De ce clasă abstractă?

1. Manipularea unui set de clase printr-o **interfață comună**
2. **Reutilizarea** unei serii de **metode** și **membri** din această clasă în **clase derivate**

Metodele suprascrise (overriding methods) -> apelate folosind **dynamic binding**

Clase ce moștenesc o clasă abstractă

- o clasă care moștenește o clasă abstractă este abstractă dacă nu implementează **toate** metodele abstracte ale clasei de bază => o clasă care poate fi **instanțiată** (nu este abstractă) și care moștenește o clasă abstractă trebuie să implementeze toate metodele abstracte pe lanțul moștenirii

Interfețe

Definiție -> colecție de **metode abstracte** (nu specifică nici o implementare pentru ele) și **declarații de constante**

- asemănător conceptului de clasă abstractă, doar că **elimină orice implementare de metode**
- protocol de comunicare
- o clasă care implementează o interfață trebuie obligatoriu să specifice **implementări** pentru **toate metodele interfeței**
- clasele pot implementa interfețe

Definire interfață

```
[public] interface NumeInterfata
[extends SuperInterfata1, SuperInterfata2...]
{
    /* Corpul interfetei:
    Declarații de constante
    Declarații de metode abstracte
    */
}
```

O interfață conține

- **constante** - pot avea modificatorii public, static și final (sunt impliciți), dar nu pot avea alți modificatori. Constantele trebuie obligatoriu inițializate!
- **metode fără implementarea** - pot avea sau nu modificatorul public (este implicit), dar nu pot avea alți modificatori

```
interface Exemplu {
    int MAX = 100;   // echivalent cu:
    public static final int MAX = 100;
    int MAX;   // Incorect, lipseste initializarea
    private int x = 1; // Incorect, modificator nepermis
    void metoda();    // Echivalent cu:
    public void metoda();
    protected void metoda2();
    // Incorect, modificator nepermis
}
```

-

Interfețe vs Clase abstracte

- extinderea unei clase abstracte forțează o relație între clase
- implementarea unei interfețe specifică doar necesitatea implementării unor anumite metode

Utilitatea interfețelor

- definirea unor **similarități** între clase independente
- impunerea unor specificații (asigură că toate clasele care implementează o interfață pun la dispoziție metodele specificate în interfață)
- definirea unor **grupuri de constante**

```
public interface Luni {
   int IAN=1, FEB=2, ..., DEC=12;
}
...
if (luna < Luni.DEC)
   luna ++;
else
   luna = Luni.IAN;
```

- **transmiterea metodelor ca parametri**

Deleted: ***Implementarea unei interfețe***¶
class NumeClasa implements NumeInterfata
sau:
class NumeClasa implements Interfata1, Interfata2, ... ¶
o clasă poate avea și alte metode și variabile membre în afară de cele definite de interfață¶
implementarea unei interfețe poate să fie și o clasă abstractă¶
modificarea unei interfețe -> modificarea tuturor claselor care implementează acea interfață

Formatted: Font: Bold, Italic

Formatted: Font: 11 pt

Interfața Comparable

```
public interface Comparable {
    int compareTo(Object o);
}
```

Metoda **compareTo** trebuie să returneze:

- **o valoare strict negativă** - dacă obiectul curent (this) este **mai mic** decât obiectul primit ca argument
- **zero** - dacă obiectul curent este **egal** cu obiectul primit ca argument
- **o valoare strict pozitivă** - dacă obiectul curent este **mai mare** decât obiectul primit ca argument

```
public boolean equals ( Object o) {
      if (!( o instanceof Persoana ))  return false ;
      Persoana p = ( Persoana ) o;
      return (cod == p.cod) && ( nume . equals (p. nume ));
}
public int compareTo ( Object o) {
      if (o = = null ) throw new NullPointerException ();
      if (!( o instanceof Persoana ))
         throw new ClassCastException ("Nu pot compara !");
      Persoana p = ( Persoana ) o;
      return (cod - p.cod);
}
```

Interfața Comparator

```
int compare(Object o1, Object o2);
```

- în cazul în care dorim să sortăm elementele unui vector ce conține referințe după alt criteriu decât ordinea naturală a elementelor

```
class MyComp implements Comparator{
  public int compare ( Object o1 , Object o2) {
    Persoana p1 = ( Persoana )o1;
    Persoana p2 = ( Persoana )o2;
    return (p1. nume . compareTo (p2. nume ));
  }
  …
  Arrays.sort(p, new MyComp());
```

sau

```
Arrays . sort (p, new Comparator () {
          public int compare ( Object o1 , Object o2) {
             Persoana p1 = ( Persoana )o1;
             Persoana p2 = ( Persoana )o2;
             return (p1. nume . compareTo (p2. nume ));
          }
} );
```

Colecții

- o colecție este un obiect care **grupează mai multe elemente** într-o singură unitate
- tipul de date al elementelor dintr-o colecție este **Object**

Collection

```
public interface Collection {
    // Metode cu caracter general
    int size();
    boolean isEmpty();
    void clear();
    Iterator iterator();
    // Operatii la nivel de element
    boolean contains(Object element);
    boolean add(Object element);
    boolean remove(Object element);
    // Operatii la nivel de colectie
    boolean containsAll(Collection c);
    boolean addAll(Collection c);
    boolean removeAll(Collection c);
    boolean retainAll(Collection c);
    // Metode de conversie in vector
    Object[ ] toArray();
    …
    }
```

Set

- mulțime în sens matematic (nu poate avea elemente duplicate)
- implementări - **HashSet**, **LinkedHashSet**, **TreeSet**

SortedSet

- mulțime de elemente sortate

Commented [2]: Am vazut in subiectele de anii trecuti probleme in care se cere definirea unei clase care implementeaza interfata Comparable

Formatted: Font: 11 pt

- ordonarea elementelor se face conform **ordinii lor naturale** sau conform cu **ordinea dată de un comparator specificat** la crearea colecției

```
public interface SortedSet extends Set {
    // Subliste
    SortedSet subSet(Object fromElement,
                     Object toElement);
    SortedSet headSet(Object toElement);
    SortedSet tailSet(Object fromElement);
    // Capete
    Object first();
    Object last();
    Comparator comparator();
}
```

HashSet

- **tabelă de dispersie** (hash table)
- calculează codul de dispersie al elementelor pe baza metodei **hashCode**, definită în clasa Object
- nu avem garanții asupra ordinii de parcurgere
- doi iteratori diferiți pot parcurge elementele mulțimii în ordine diferită

LinkedHashSet : tabelă de dispersie.

Diferența față de HashSet - menține o listă dublu-înlănțuită peste toate elementele sale. => elementele rămân în **ordinea** în care au fost inserate.

raise you am

Static / dynamic Binding

Legare statică/ dinamică

Binding = procesul de a stabili ce metodă sau variabilă va fi apelată

STATIC BINDING	*DYNAMIC BINDING*
se realizează la **Compilare**	se realizează la **Rulare - Execuție**
este utilizat de metode și variabile **private**, **final** sau **static**	este utilizat de metode **virtuale - abstracte**
metode supraîncărcate **overloaded methods**	metode supradefinite **overriding methods**
variabilele membru prezintă static binding, deoarece Java nu permite comportament polimorfic pentru variabilele membru	este **mai lent** pt. că apare în momentul execuției și necesită timp pentru a determina care metodă este apelată

More about dynamic binding

- metodele suprascrise (override) în clasele derivate vor fi apelate folosind dynamic binding
- dynamic binding este un mecanism prin care compilatorul, în momentul în care nu poate determina implementarea unei metode în avans, lasă la latitudinea JVM-ului (maşinii virtuale) alegerea implementării potrivite, în funcţie de tipul real al obiectului

Exemple Static Binding vs Dynamic Binding

```
public class Shape {
    int x=10;
    void draw() {
        System.out.println("Shape");
    }
}
public class Circle extends Shape{
    int x=5;
    void draw(){
        System.out.println("Circle");
    }
}
public class Test{
    public static void main(String[] args) {
        Shape shape = new Circle();
        shape.draw();  // DYNAMIC BINDING
        System.out.println(shape.x);  // STATIC BINDING
    }
}
```

// shape.x = valoarea lui x din Shape (10)

Commented [3]: a fost subiect la POO in 2019

Formatted: Font: 11 pt